BACHELOROFCOMPUTER APPLICATIONS (BCA)

(Revised Syllabus)

BCA(Revised Syllabus)/ASSIGN/SEMESTER-III

ASSIGNMENTS

(July-2024&January-2025)

MCS-021,MCS-023,MCS-014,BCS-031,BCSL-032,BCSL-033,BCSL-034,

SCHOOLOFCOMPUTERANDINFORMATIONSCIENCES
INDIRA GANDHI NATIONAL OPEN UNIVERSITY
MAIDAN GARHI, NEW DELHI – 110 068

इग्नू
जन-जन का
विश्वविद्यालय

ignou
THE PEOPLE'S
UNIVERSITY

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

Assignment Submission for Term-End Exam December -2024

ENROLLMENT NUMBER : 2 3 5 0 3 6 1 0 0 4

NAME OF THE STUDENT : Deepak Kumar Singh

STUDENT ADDRESS : House No - 395/103, Uttam nagar - 110059

PROGRAMME TITLE & CODE : BCA: Bachelor of Computer Application

COURSE TITLE : BCS-031

COURSE CODE : Programming in C++ Assigment

REGIONAL CENTRE NAME & CODE : 38: Delhi - 3 Nasaina

STUDY CENTRE NAME & CODE : 38046: Rajdhani Collage Raja Gardem

MOBILE NUMBER : 7 3 0 3 9 3 8 4 5 0

E-MAIL ID : Kashyap04-0098@gmail.com

DATE OF SUBMISSION: .....

(SIGNATURE OF THE STUDENT)

Course Code	:	BCS-031
Course Title	:	Programming in C++Assignment
Number	:	BCA(III)031/Assignment/2024-25
Maximum Marks	:	100
Weightage	:	25%
Last Date of Submission	:	31st October,2024(for July session) 30th April,2025(for January session)

This assignment has three questions carrying a total of 80 marks. Answer all the questions. Rest 20 marks are for viva-voce. You may use illustrations and diagrams to enhance explanations. Please go through the guidelines regarding assignments given in the Programme Guide for the format of presentation. Wherever required, you may write C++ program and take its output as part of solution

Q1. What is a Virtual Constructor? What are its advantages? Explain with examples. **(30 Marks)**

Q2. What is a Virtual Function? How does it differ from a Pure Virtual Function? Explain with examples. **(30 Marks)**

Q3. What is a Header File? Explain any 5 header files along with their functions. **(20 Marks)**

TOPIC _____ DATE _____

Q-1 What is virtual constructor? What are its advantages? Explain with examples.

Ans (1) Virtual constructor through factory Method Pattern

In this content, the virtual constructor refers to a mechanism that allow the creation of object based on runtime information, Making the object instantion process more flexible. the factory Method Pattern is commonly used to achieve this.

Example (Factory Method in C++):

```
# include <iostream>
using namspace Std;

// Base class
Class Animal {
Public :
        Virtual void Sound () = 0;
        Static Animal* Create (const String &type)
};

// Derived class 1
class Dog : Public Animal {
Public :
        void Sound () orerride {
            Count << "Bark!" << end];
        }
};
```

TOPIC DATE

```
// Derived class 3
class Cat : Public Animal {
Public:
        void Sound() override {
            cout << "Meow!" << endl;
        }
};

// Factory Method (virtual constructor)
Animal* Animal::create(const string &type) {
    if (type == "Dog")
        return new Dog();
        else if (type == "cat")
            return New cat();
        else
            return Nullptr;
}

int main() {
    Animal* dog = Animal::Create("Dog");
    Dog->sound(); // Output: Bark!

    Anincl* cat = Animal::create("cat");
    cat->sound(); // output: Meow!

    Delete dog;
    Delete cat;

    return 0;

}
```

TOPIC _____ DATE _____

- Advantages:

1. Runtime Polymorphism: the actual object type (eg Dog or cat) can be determined at runtime, Making the code more flexible and extensible.

2. Decoupling: the client code doesn't need to know the exact type of the object being created which reduces dependancies between classes.

3. Single Responsibility: the object creation logic is centralized, adhering to the single Responsibility principle by decoupling the instaition from business logic.

4. Encapsulation of object creation: the creation logic is Hidden, which leads to better encapsulation

2 Virtual Constructor through Prototype Pattern

Another approach for implementing a virtual Constructor is the Prototype Pattern, where an existing object is cloned to create new instances.

Example (Prototype Pattern in C++)

```
# include <iostream>
  using namespace std;
```

```
// Base class
class Animal {
Public:
  virtual Animal* clone() const =0;
  virtual void sound() const =0;
};

// Derived class 1
Class Dog : public Animal {
Public:
    Dog* clone() const override {
      return new Dog(*this);
    }
    void sound() const override {
      cout << "Bark!" << endl;
    }
};

// Derived class 2
Class cat : Public Animal {
Public:
    cat* clone() const override {
        return new cat(*this);
    }
    void sound() const override {
      count << "Meow" << endl;
    }
};

int main() {
```

TOPIC _____ DATE _____

```
Animal *originalDog = new Dog();
Animal *cloneDog = originalDog->clone();

Original Dog->sound();
cloned Dog->Sound();

Delete originalDog;
Delete clonedDog;

return 0;

}
```

- Advantage of Prototype Pattern:

(1) Object Duplication : Allow cloning of existing object intsedted of creating them from Scratch Saving construction time and resources.

(2) Dynamic System : Enable the system to create New object dynamically based on prototyped at runtime.

(3) Encapsulation : the cloning process encapsulates the instantition logic whithin the class leading to acleamer degin.

TOPIC _____ DATE _____

Q-2 What is a virtual function? How does it differ from a Pure Virtual Function? Explain with Examples.

Ans (1) Virtual Function

A virtual function is a function that is defined in a base class and is intended to be overridden by derived classes. the base class provied an implementation of the function, but derived classes can override it to provied their own specific behavior. the key feature is running polymorphism, where the function that gets called depends on the actual type of the object, even if it's referrend by a base class pointer or reference.

Example of virtual function:

```
#include <iostream>
using namespace std;

class Animal {
Public:
    //virtual function
    virtual void sound() {
        cout << "Animal make a sound" << endl;
    }
};

Class Dog : public Animal {
Public:
```

TOPIC _____ DATE _____

```
// overriding virtual function
void sound() override {
    cout << "Dog barks" << endl;
  }
};

Class Cat : Public Animal {
Public
    // overriding virtual function
    void sound() orride {
        cout << "cat meows" << endl;
    }
};

int main() {
    Animal * animal1 = new Dog();
    Animal * animal2 = new Cat();

// Runtime ~~pto~~ polymorphism
animal1 -> Sound();
animal2 -> Sound();

delete animal1;
delete animal2;

return 0;

}
```

TOPIC DATE

- Characteristic of virtual functions:

1. Defined in Base Class : the function is declared as virtual in the base class.

2. Provides a Default implementation : the base class usually provides a default implementation that may not be overriden.

3. Overridden in Derived class : Derived classes can provide their own implementation

4. Runtime Polymorphism : when a derived class object is pointed to by a base class pointer the derived class version of the function in invoked.

2. Pure virtual Function

A pure virtual function is a virtual function in the base class that does not provide implementation. it forces derived class to overrider the function.

- Syntax of Pure virtual Function

Virtual Function

```
Virtual void functionname () = 0;
```

```
#include <iostream>
using namespace std;

class Animal {
Public:
    // Pure virtual function
    virtual void sound() = 0; // No implemention
};

class Dog : Public Animal {
Public:
    // overriding pure virtual function
    void sound() override {
        cout << "Dog barks" << endl;
    }
};

class cat ; Public Animal {
Public:
    // overriding pure virtual function
    void sound() orreside {
        cout << "cat meow" << endl";
    }
};

int main() {
    Animal* animal1 = new Dog();
    Animal* animal2 = new cat();
```

TOPIC _____ DATE _____

```
delete animal1;
delete animal2;

return 0;

}
```

- Characteristics of Pure virtual functions:

(1) No implementation in Base class : the base class does Not Proide an implementation For a pure virtual function. the function is declared but Set equal to 0, meaning it is abstract.

(2) Must Be overridden : Derived classes are required to provied their own implementation of an pure virtual function

(3) Base class is Abstract : Any class that Contain a pure virtual function becomes an abstract class and cannot be instaintiated.

TOPIC _____ DATE _____

Q-3 What is a Header File ? Explain any 5 Header Files along with their Functions ?

Ans A Header files is a file for C++ programming that typically contain function declaration, macros constants and sometimes inline function. the main purpose of a Header file is to allow code reusable and to separate the function declaration.

- Commonly used Header files and their functions:

(1) <iostream> (input/output stream)

This is one of the most widely used Header files in C++ for Handling input and output operation it contains the defination for the standard input/output steam object such as cin, cout, and cerr.

- Key Components:

- std::cin : Reads input from the standard input (usually the keyboard)
- Std::cout : Prints output to the standard output (usually the console)
- std::cerr : outputs error message to the console.

Example:

```
#include <iostream>
#using namespace std;
```

TOPIC _____ DATE _____

```
int main () {
    count << " Enter a number : ");
    int num;
    cin >> num;
    cout << "you entered : " << num << endl;
    return 0;
}
```

2 <Cmath> (c++ Math Library)

the <Cmath> Header file provide a collection of function For performing mathematical operation such as trigonometic, logarithmic, and exponential calculation it is the C++ version of the C library <math.h>

- Key Functions

- Sqrt() : Computes the Square root of a number

- Pow () : Raise a number to a special power

- Sin (), cos(), tan() : Trigonometic functions.

- log () : Computes the Natural logarithm of Number.

```
#include <iostream>
#includ <Cmath>
```

TOPIC _____ DATE _____

```
int main() {
    double x = 9.0;
    std::cout << "Square root of 9 is". << sqrt(x)
    << std::endl;

return 0;

}
```

(3) <vector> (STL Vector)

the <vector> Header files is part of the C++ standard template (STL) and provide the std::vector class, which is a dynamic array.

- Key function:

- Push_back(): Adds an element to the end of vector
- Size() : Returns the number of elements in the vector
- at() : Accesses an element at a given idex with bound checke
- Clear(): Remove all element from the vector.

```
#include <iostream>
#include <vector>
```

TOPIC _____ DATE _____

```
int main () {
    std::vector <int> numbers;
    number.push_back (10);
    number.push_back (20);
    number.push_back (30);

std::cout << "vector size: " << number, size () << stud::endl
Stud::count << " First element: " << number.at (0) << Stud::endl

return 0;

}
```

(4) <string> (C++ string class)

the <string> Header file provide the std::string class which allows the manipulation of sequences of characters (string) in C++ in a more intutive and dynamic way than C-style character arrays.

- Key Function
- length() : Return the length of string.
- substr : Extraits a substring.
- Find : Finds a substring with a string.
- append : Appends another string to the current string.

TOPIC DATE

```
#include <iostream>
#include <string>

int main() {
    std::string str = "Hello,world";
    std::cout << "length of string:" << str.length()
    << std::endl;
    std::cout << "substring:" << str.substr(0,5)
    << std::endl;

  return 0;

}
```

5. <algorithm> (STL Algorithms)

the <algorithm> header file provides a collection of function that perform algorithm operation such as searching sorting, and manipulating data structures like arrays or vectors. it is part of the C++ standard template library (STL).

- Key Function:

- Sort() : sorts elements in a range

- Find() : Find an element in a range

- reverse() : Reverses a sequence.

- min_element(), max_element : Find the minimum or maximum element in a range.

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numbers = {5,3,9,1,7};

    std::sort(numbers.begin(), numbers.end());
    std::cout << "Sorted number : ";
    for (int num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

  return 0;

}
```